

AI SW·어플리케이션 품질검증 특강 · 1일차

AI 품질검증의 이해

소프트웨어 테스트의 기초 원리와 AI 모델 품질 평가 지표

2026. 10. 1. (목) 09:00 ~ 18:00

강사 소개

이재희

현재

- 제이소프트웨어(주) 대표이사 (2017~)
- 빅데이터·인공지능 프로젝트, AX 엔지니어링, 에이전틱 AI 개발
- 스타트업 고객사 MVP 다수 기획·개발 (헬스케어, 뷰티, 스포츠 등)
- 한양대학교 공학대학원 컴퓨터공학 재학

주요 프로젝트

- 신한카드 빅데이터·AI 플랫폼 구축, 초개인화 (2018~2019)
- SK하이닉스 GPU 도커 대여 플랫폼, AI 실무 강의 (2017~2018)
- 정보화진흥원 K-ICT 빅데이터 센터 인프라 (2014~2016)
- 삼성전자 BDP2.0 빅데이터 플랫폼 (2013~2014)
- LG전자 그룹웨어 경량화 아키텍처 (2021)

강사 소개

현장에서 만들고, 현장에서 가르칩니다

기간	과정
2025.09~2026.04	kt cloud TECH UP 풀스택 과정 (866시간)
2021~2025	멀티캠퍼스 채용연계 풀스택 개발자 부트캠프
2024~2025	강남구 혁신인재육성 아카데미 생성형 AI 기반 웹서비스 전문가 양성과정
2021~2023	데이터 기반 인공지능 시스템 엔지니어 양성과정
2023~2024	클라우드 아카데미 (AWS·하이브리드 클라우드)
2017~2018	SK하이닉스 TensorFlow 기반 인공지능 실무 강의

MVP를 만들고 운영해 온 경험으로, AI 서비스를 믿고 쓸 수 있게 만드는 방법을 함께 봅니다.

오늘의 흐름

1일차 시간표

교시	시간	주제	실습
1	09:10~10:00	왜 AI 품질검증인가 + 테스트 기초 원리	
2	10:10~11:00	블랙박스·화이트박스 테스트, 테스트 단계	미니 실습
3	11:10~12:00	분류 모델 평가 지표	노트북 01
4	13:00~13:50	회귀 모델 평가 지표	노트북 02
5	14:00~14:50	AI 품질 이슈 ①: 과적합과 데이터 누수	노트북 03
6	15:00~15:50	AI 품질 이슈 ②: 데이터 편향과 강건성	노트북 04
7	16:00~16:50	데이터 품질 관리: 정합성 검증과 검수 기준	노트북 05
8	17:00~17:50	[실습] 검수 기준서 작성과 평가 결과 해석	조별 발표

점심 12:00~13:00 · 수업 50분, 쉬는 시간 10분

이틀을 관통하는 질문

이 AI 모델, 믿고 써도 될까요?

1일차는 판단의 기준을, 2일차는 그 기준을 자동으로 확인하는 방법을 배웁니다.

이틀 동안 배우는 것

1일차 · 기준을 이해한다

- 테스트 원리와 기법
- 분류·회귀 평가 지표
- 과적합·편향·강건성
- 데이터 품질 관리

▶ 데이터 품질 검수 기준서

2일차 · 기준을 자동화한다

- unittest · PyTest
- 교차 검증 · 성능 리포트
- GitHub Actions CI
- 이슈 트래커 버그 리포트

▶ 테스트·CI·버그 리포트가 담긴 저장소

실습 자료: github.com/leejaehee1/ai-quality-lab · 실습 환경: Google Colab

1교시 · AI는 어떻게 실패하는가

모델의 실패는 회사의 책임이 됩니다

채용 AI · 2018년 보도

과거 10년 이력서로 학습한 평가 모델이 여성 지원자에게 불리하게 점수를 매겨 폐기

→ 6교시 데이터 편향

주택 가격 예측 · 2021년

가격 예측 모델로 집을 사고팔던 사업이 시장 변화 속 예측 실패로 철수

→ 4교시 회귀 지표, 6교시 분포 변화

항공사 챗봇 · 2024년

실제와 다른 환불 규정을 안내해 회사가 배상

→ 출력 검증. 생성형 AI는 부록

출시 전에 무엇을 확인했어야 했을까요?

채용 AI

과거 데이터에 담긴 편향을 어떻게 미리 알 수 있었을까?

주택 가격 예측

시장이 바뀌어도 예측이 맞는지 어떻게 확인할 수 있었을까?

항공사 챗봇

모델이 틀린 정보를 내놓기 전에 무엇을 막을 수 있었을까?

오류 → 결함 → 장애

용어	뜻	ML 예시
오류 (Error)	사람의 실수	전처리 코드에서 나이 단위를 '개월'로 잘못 이해
결함 (Defect, Bug)	산출물(코드·데이터·문서)에 남은 잘못	나이 컬럼을 12로 나누지 않은 전처리 함수
장애 (Failure)	실행 중 드러난, 기대와 다른 동작	모델이 20대 고객을 모두 고위험으로 예측

사람의 실수가 산출물에 결함을 남기고, 결함이 실행되면 장애가 됩니다.

결함을 찾고, 품질에 대한 정보를 준다

테스팅의 목적

- 결함을 찾는다
- 품질에 대한 정보를 제공해 출시 결정을 돕는다
- 요구사항을 만족하는지 확인한다

QA와 테스트

- QA(품질 보증): 좋은 프로세스로 결함을 예방
- 테스트: 결과물에서 결함을 찾아냄
- 예: 리뷰 규칙 만들기(QA) · 코드 실행해 버그 찾기(테스팅)

결함이 숨는 곳이 세 군데입니다

코드

전처리, 학습·예측 파이프라인, 배포 코드

데이터

결측, 범위 이탈, 중복, 라벨 오류, 편향, 누수

모델

성능 부족, 과적합, 그룹별 성능 차이, 강건성 부족

코드가 완벽해도 데이터가 틀리면 모델이 틀립니다.

테스팅에 대한 일곱 가지 사실

#	원칙	AI 예시
1	테스팅은 결함이 있음을 보여 줄 뿐, 없음을 증명하지 못한다	테스트 정확도 99%여도 실제 환경에서 틀릴 수 있다
2	완벽한(전수) 테스트는 불가능하다	이미지 입력은 사실상 무한 → 위험한 영역부터
3	조기 테스트가 시간과 비용을 아낀다	학습 전에 데이터를 검수하면 재학습 비용을 아낀다
4	결함은 몰려 있다	소수 그룹, 드문 클래스에서 오류가 몰린다

테스트도 닳고, 정황에 따라 다릅니다

#	원칙	AI 예시
5	테스트는 닳는다 (살충제 패러독스)	같은 테스트셋에 반복해서 맞추면 테스트셋에 과적합된다
6	테스팅은 정황에 의존한다	암 진단은 재현율, 스팸 필터는 정밀도가 중요하다
7	결함 부재의 궤변	정확도는 높지만 업무에 쓸모없는 모델

5번은 예전 강의계획서의 '살충제 패러독스'를 v4.0에서 'Tests wear out'으로 표현한 것입니다.

계획부터 완료까지, 반복하며 진행합니다

활동	질문	ML 프로젝트에서
계획	무엇을, 언제까지, 어떤 기준으로?	재현율 0.9 이상, 그룹 차이 0.05 이하면 출시
분석	무엇을 테스트할까?	데이터 스키마, 성능 하한, 그룹별 성능, 강건성
설계	어떻게 테스트할까?	경계값, 결정 테이블, 섭동 테스트
구현	실행 준비	테스트 데이터·코드 작성 (2일차)
실행	실행하고 기록	pytest 실행, 성능 리포트 생성 (2일차)
완료	정리와 교훈	결함 보고, 출시 판단 (2일차)

모니터링과 제어는 전 과정에 걸쳐 함께 진행합니다.

개발 단계마다 짝이 되는 테스트 단계

개발 단계	짝이 되는 테스트
요구사항 정의	인수 테스트
시스템 설계	시스템 테스트
상세 설계	통합 테스트
구현(코딩)	유닛 테스트

핵심

테스트 설계는 개발 단계에서 미리 시작한다. 요구사항을 정할 때
인수 기준도 함께 정한다(원칙 3, 조기 테스트).

데이터 · 모델 · 코드를 모두 봅니다

대상	무엇을 확인하나	이 특강에서
데이터	스키마, 결측, 범위, 중복, 편향, 누수	1일차 7교시, 2일차 3교시
모델	성능 지표, 과적합, 그룹별 성능, 강건성	1일차 3~6교시, 2일차 4교시
코드·인프라	전처리 함수, 파이프라인, 배포	2일차 1~2교시, 6교시

참고: Google 연구진의 "The ML Test Score"(2017)는 데이터·모델·인프라·모니터링 네 영역의 점검 목록을 제안했습니다.

이 교시 내내 이 함수를 테스트합니다

```
is_eligible(age, months_resident, employed)
```

- 나이: 만 19세 이상 34세 이하
- 해당 지역 거주 6개월 이상
- 미취업 상태
- 세 조건을 모두 만족하면 True
- 나이가 0~120 범위 밖이거나 숫자가 아니면 ValueError

블랙박스 vs 화이트박스

구분	블랙박스 (명세 기반)	화이트박스 (구조 기반)
보고 설계하는 것	요구사항·명세	코드의 내부 구조
대표 기법	동등 분할, 경계값, 결정 테이블, 상태 전이	구문 테스트, 분기 테스트
측정	요구사항 커버리지	구문·분기 커버리지
주로 누가	테스터, 현업	개발자
ML에서	모델에 입력을 넣고 예측을 확인	전처리·후처리 코드의 모든 분기 실행

ML 모델 자체는 주로 블랙박스로, 모델 주변 코드는 화이트박스로 테스트합니다. 세 번째로 경험 기반 테스트(오류 추정, 탐색적 테스트)가 있습니다.

같이 처리될 입력을 묶고 대표값 하나씩

분할	범위	대표값	기대 결과
무효: 범위 밖(작음)	$\text{age} < 0$	-5	ValueError
유효: 나이 미달	$0 \leq \text{age} < 19$	10	False
유효: 대상 나이	$19 \leq \text{age} \leq 34$	25	True (다른 조건 충족 시)
유효: 나이 초과	$34 < \text{age} \leq 120$	50	False
무효: 범위 밖(큼)	$\text{age} > 120$	200	ValueError
무효: 숫자 아님	문자열 등	"스물"	ValueError

무효 분할도 반드시 테스트합니다. 커버리지 = 테스트한 분할 수 ÷ 전체 분할 수

결함은 경계에 몰립니다

방식	경계 19에서	경계 34에서	전체 테스트 값
2값 경계값	18, 19	34, 35	18, 19, 34, 35
3값 경계값	18, 19, 20	33, 34, 35	18, 19, 20, 33, 34, 35

- < 와 <= 를 헷갈리는 실수를 잡습니다
- ISTQB v4.0은 2값·3값 방식을 모두 다룹니다
- 범위 밖 경계(-1, 0, 120, 121)도 같은 방식으로

모델 입력에도 경계가 있습니다

학습 범위 밖 입력

학습 데이터의 나이가 20~60세였다면, 19세나 61세는 모델이 한 번도 보지 못한 영역입니다.

그래서

범위 밖 입력은 거부하거나 경고하는 규칙이 필요합니다. 6교시 강건성, 7교시 범위 검증으로 이어집니다.

조건의 조합마다 결과를 정합니다

규칙	R1	R2	R3	R4	R5~R8
C1 나이 19~34	T	T	T	T	F
C2 거주 6개월 이상	T	T	F	F	—
C3 미취업	T	F	T	F	—
결과: 대상	예	아니오	아니오	아니오	아니오

- 조건 3개 → $2^3 = 8$ 개 규칙. C1이 F이면 나머지와 무관하므로 R5~R8을 하나로 줄일 수 있음
- 중요한 기능은 줄이지 않고 전체 규칙을 테스트 (조건 순서 구현 실수 대비)

평가 데이터를 골고루 뽑는 사고방식

'성별 × 연령대 × 지역' 조합마다 테스트 데이터가 충분히 있는지 확인하는 표가 곧 결정 테이블입니다. 조합 중 데이터가 거의 없는 칸이 있다면, 그 집단의 성능은 믿을 수 없습니다(6교시 그룹별 평가).

코드를 보고 테스트합니다

```
def is_eligible(age, months_resident, employed):  
    if not isinstance(age, (int, float)) or not (0 <= age <= 120):    # 결정 D1  
        raise ValueError("invalid age")                                # 문장 S1  
    result = False                                                    # 문장 S2  
    if 19 <= age <= 34 and months_resident >= 6 and not employed:      # 결정 D2  
        result = True                                                  # 문장 S3  
    return result                                                      # 문장 S4
```

- 구문 커버리지 = 실행된 문장 ÷ 전체 실행 가능 문장
- 분기 커버리지 = 실행된 분기(결정의 참·거짓) ÷ 전체 분기

구문 100%여도 분기는 부족할 수 있습니다

테스트	실행된 문장	실행된 분기
T1: is_eligible(25, 12, False)	S2, S3, S4	D1-거짓, D2-참
T2: is_eligible(200, 12, False)	S1	D1-참
T3: is_eligible(40, 12, False)	S2, S4	D1-거짓, D2-거짓

실행한 테스트	구문 커버리지	분기 커버리지
T1	3/4 = 75%	2/4 = 50%
T1 + T2	4/4 = 100%	3/4 = 75%
T1 + T2 + T3	100%	100%

분기 100%여도 결함은 남을 수 있습니다

분기 $\supset$ 구문

분기 커버리지 100%이면 구문 커버리지도 100%입니다. 반대는 성립하지 않습니다.

결함이 숨는 경우

$19 \leq \text{age}$ 를 $18 \leq \text{age}$ 로 잘못 써도 T1·T2·T3는 모두 통과합니다. 경계값 18을 테스트해야 잡힙니다.

그래서 화이트박스과 블랙박스를 함께 씁니다. 2일차에 `pytest-cov`로 커버리지를 자동 측정합니다.

작은 단위부터 전체까지

단계	대상	ML 파이프라인 예	주로
유닛 (컴포넌트)	함수·클래스 하나	fill_missing(), scale_features()	개발자
통합	모듈 사이 연결	전처리 출력이 모델 입력 형식과 맞나, 모델 저장·불러오기	개발자
시스템	전체를 사용자 관점에서	예측 API 요청 → 응답 형식·시간·결과	테스터
인수	업무 요구사항 충족	재현율 0.9 이상, 응답 1초 이내를 만족하나	현업·고객

ISTQB v4.0은 컴포넌트 → 컴포넌트 통합 → 시스템 → 시스템 통합 → 인수로 더 세분합니다.

빠른 테스트는 많이, 느린 테스트는 핵심만

E2E · 시스템

적게. 느리고 비쌈

통합

중간

유닛

많이. 빠르고 싼

ML에서는 '학습 후 성능 확인'이 느립니다. 작은 데이터로 빠르게 도는 테스트와 전체 데이터 테스트를 나눕니다(2일차 CI).

허용되지 않는 전이가 막히는가

[학습 완료] --검증 통과--> [스테이징] --승인--> [운영] --성능 저하--> [롤백]
└--검증 실패--> [폐기]

- 모든 상태를 거치는 테스트
- 모든 전이를 거치는 테스트
- 금지된 전이(학습 완료 → 바로 운영)가 막히는지 테스트

선택 2교시 · 미니 실습

직접 풀어 봅시다

Q1

대출 금액 100만 원 이상 5,000만 원 이하. 3값 경계값을 모두 적으세요. (단위 1원)

Q2

앞의 코드에서 T2 + T3만 실행하면 구문·분기 커버리지는 각각 몇 %?

Q3

'VIP 여부 × 쿠폰 보유 여부'로 할인율을 정할 때 결정 테이블 규칙은 몇 개?

1,000명 선별 검사: 환자 60명, 정상 940명

	예측: 환자 (양성)	예측: 정상 (음성)
실제: 환자	TP = 40 (맞게 찾음)	FN = 20 (놓침)
실제: 정상	FP = 10 (잘못 경보)	TN = 930 (맞게 넘김)

- T/F: 예측이 맞았나 틀렸나
- P/N: 모델이 양성이라고 했나 음성이라고 했나

3교시 · 먼저 정할 것

무엇이 '양성'인가

양성은 '좋은 것'이 아니라 우리가 찾으려는 대상입니다.

질병 선별

양성 = 환자

사기 탐지

양성 = 사기 거래

스팸 필터

양성 = 스팸

이 정의를 먼저 정하지 않으면 지표가 뒤집힙니다. 조금 뒤 실제 데이터에서 부딪힙니다.

같은 결과, 다른 질문

지표	공식	예제 값	질문
Accuracy (정확도)	$(TP + TN) / \text{전체}$	$970 / 1000 = 0.970$	전체 중 몇 개 맞췄나?
Precision (정밀도)	$TP / (TP + FP)$	$40 / 50 = 0.800$	환자라고 한 것 중 진짜는?
Recall (재현율)	$TP / (TP + FN)$	$40 / 60 = 0.667$	진짜 환자 중 찾아낸 비율은?
F1-score	$2PR / (P + R)$	0.727	정밀도와 재현율의 균형은?

정확도 97%인데, 환자 3명 중 1명을 놓치고 있습니다.

낮은 쪽에 끌려가도록

예: 정밀도 1.0, 재현율 0.1

- 산술 평균 = 0.55
- 조화 평균(F1) = 0.18

F1 다른 공식

$F1 = 2TP / (2TP + FP + FN) = 80 / 110 = 0.727$. TN이 공식에 없어서, 음성이 아주 많은 데이터에서 정확도보다 정직합니다.

특이도(Specificity) = $TN / (TN + FP) = 0.989$. 의료 분야에서 민감도(재현율)와 짝으로 씁니다.

3교시 · 정확도의 함정

무조건 '정상'이라고만 하는 모델

0.94

정확도

0

재현율: 환자를 한 명도 못 찾음

- 양성인 드문 문제(질병, 사기, 불량, 이탈)에서는 아무것도 안 하는 모델이 좋아 보입니다
- 먼저 클래스 비율을 보고, 기준선 모델(DummyClassifier)보다 얼마나 나은지 말합니다

더 나쁜 실수가 무엇인지로 정합니다

상황	더 나쁜 실수	중시할 지표
암 선별 검사	환자를 놓침 (FN)	재현율
스팸 필터	중요한 메일을 스팸함으로 (FP)	정밀도
금융 사기 탐지	둘 다 비쌈	F1 또는 비용 기반 판단
불량품 검사	불량품 출하 (FN)	재현율 + 정밀도 하한

품질 기준은 '재현율 0.9 이상 그리고 정밀도 0.7 이상'처럼 두 지표를 함께 겁니다.

유방암 진단 데이터 (569건, 특성 30개)

```
X, y = load_breast_cancer(return_X_y=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=42, stratify=y)
model = make_pipeline(StandardScaler(), LogisticRegression(max_iter=1000))
model.fit(X_train, y_train)
print(confusion_matrix(y_test, model.predict(X_test), labels=[0, 1]))
```

악성 기준 (테스트 143건)	TP	FN	FP	TN	정확도	정밀도	재현율	F1
결과	52	1	1	89	0.986	0.981	0.981	0.981

3교시 · 실무에서 자주 틀리는 곳

이 데이터는 0 = 악성, 1 = 양성 종양

sklearn 기본값

1을 양성 클래스로 계산 → `precision_score(y, pred)`는 '양성 종양을 찾는 정밀도' (0.989)

올바른 계산

악성을 찾으려면 `pos_label=0` 지정 → 정밀도·재현율 0.981

의학의 '양성(benign)'과 분류의 '양성 클래스(positive)'가 반대입니다.

확률을 어디서 자를지는 업무 판단입니다

임계값 (악성 확률)	악성 예측 건수	정밀도	재현율
0.9	46	1.000	0.868
0.7	50	1.000	0.943
0.5 (기본)	53	0.981	0.981
0.3	58	0.897	0.981
0.1	64	0.828	1.000

임계값을 낮추면 재현율↑ 정밀도↓. 테스트 143건 중 악성 53건 기준, 노트북 01 결과

macro · weighted · micro

방식	계산	언제
macro	클래스별 지표의 단순 평균	작은 클래스도 똑같이 중요할 때
weighted	샘플 수로 가중 평균	전체 성능을 대표할 때
micro	전체 TP·FP·FN을 합쳐 계산	단일 라벨 다중 분류에서는 정확도와 같음

macro와 weighted의 차이가 크면 작은 클래스를 의심하세요.

임계값과 무관한 구분 능력

- 임계값을 0→1로 바꾸며 재현율(TPR)과 거짓 양성률($FPR = FP / (FP + TN)$)을 그린 곡선
- AUC 1.0 = 완벽, 0.5 = 무작위 수준. 노트북 01 모델의 ROC-AUC는 0.998
- 양성이 아주 드물면 ROC-AUC가 낙관적 → 정밀도-재현율 곡선과 평균 정밀도(AP)를 함께

선택 3교시 · 테스트셋 크기와 지표의 흔들림

같은 90%, 다른 믿음

20건 중 18건 정답

정확도 90%, 95% 신뢰구간(월슨) 약 70% ~ 97%

200건 중 180건 정답

정확도 90%, 95% 신뢰구간 약 85% ~ 93%

지표 옆에 항상 테스트 건수를 함께 봅니다. 2일차 교차 검증에서 '평균 ± 표준편차'로 보는 이유입니다.

세 문제

Q1

TP=30, FP=20, FN=10, TN=940일 때
정밀도와 재현율은?

Q2

사기 거래가 1%인 데이터에서 '모두 정상'
모델의 정확도는?

Q3

공항 보안 검색에서 더 중요한 지표는?

회귀는 숫자를 예측합니다

잔차(오차)

실제값 - 예측값

양수: 모델이 낮게 예측 / 음수: 높게 예측

회귀 지표는

잔차를 어떻게 모으느냐의 차이입니다. 절댓값 평균, 제곱 평균, 설명된 비율.

잔차를 모으는 세 가지 방법

지표	공식	단위	특징
MAE (평균 절대 오차)	$ 잔차 $ 의 평균	예측 대상과 같음	직관적, 이상치에 덜 민감
MSE (평균 제곱 오차)	잔차 ² 의 평균	단위의 제곱	큰 오차를 크게 벌줌, 학습 손실로 많이 씀
RMSE (평균 제곱근 오차)	$\sqrt{MSE}$	예측 대상과 같음	MSE를 원래 단위로

잔차 5개 (단위: 만 원)

경우	잔차	MAE	MSE	RMSE
A: 큰 오차 1건	2, -1, 3, -2, 8	3.2	16.4	4.05
B: 고르게 틀림	2, -1, 3, -2, 2	2.0	4.4	2.10

큰 오차 하나에 MAE는 1.6배, MSE는 3.7배. RMSE가 MAE보다 훨씬 크면 '가끔 크게 틀린다'는 신호입니다.

선택 4교시 · 어느 지표를 쓸까

항상 둘 다 보고, 차이를 해석합니다

RMSE 중심

큰 오차가 특히 위험할 때. 재고 부족으로 공장이 멈추는 수요 예측

MAE 중심

오차 비용이 크기에 비례할 때, 이상치 데이터가 섞여 있을 때

평균으로 찍는 것보다 얼마나 나은가

R ² 값	뜻
1.0	완벽한 예측
0.0	항상 평균값으로 예측하는 것과 같은 수준
0보다 작음	평균으로 찍는 것보다도 못함 (가능합니다)

- $R^2 = 1 - (\text{잔차 제곱합} \div \text{평균과의 차이 제곱합})$
- 단위가 없어 대략 비교가 쉽지만, R^2 가 높아도 오차가 업무 허용치를 넘을 수 있음 → MAE·RMSE와 함께

당뇨 진행도 예측 (442명, 특성 10개)

모델	MAE	RMSE	R ²
기준선 (학습 데이터 평균으로 예측)	65.5	74.9	-0.014
선형 회귀	41.5	53.4	0.485

목표값 범위 25 ~ 346, 평균 152

숫자만 말하지 말고 맥락을 함께

나쁜 보고

"RMSE는 53.4입니다."

좋은 보고

"진행도(25~346, 평균 152)를 예측할 때 평균 약 41 틀립니다. 가끔 크게 틀려 RMSE는 53.4입니다. 평균으로 찍는 것보다 낮지만 변동의 절반 정도만 설명합니다(R^2 0.49)."

비율로 말하기 쉽지만 0 근처에서 폭발합니다

- $MAPE = |잔차 \div 실제값|$ 의 평균. '평균 12% 틀린다'처럼 말하기 쉬움
- 당뇨 데이터 MAPE: 0.373
- 실제값 1을 6으로 예측한 한 건이 섞이면 MAPE 1.026 (100% 이상)
- 판매량 0인 날이 있는 데이터에는 쓰지 않습니다

x축 예측값, y축 잔차

모양	뜻
0 주변에 고르게 흩어짐	좋음. 모델이 놓친 패턴 없음
곡선 모양	비선형 관계를 놓침
오른쪽으로 갈수록 퍼짐	값이 클수록 오차 증가 → 로그 변환 등 검토
특정 점들이 멀리 떨어짐	이상치 또는 데이터 오류 → 7교시 데이터 검수

4교시 · 확인 퀴즈

직접 계산해 봅시다

잔차가 [1, -1, 1, -1, 10]일 때 MAE, MSE, RMSE는?

노트북 02 마지막 셀에서 코드로 확인할 수 있습니다.

외운 것과 이해한 것의 차이

상태	학습 성능	테스트 성능	비유
과소적합	낮음	낮음	공부를 너무 적게 함
적절	높음	높음 (학습과 비슷)	원리를 이해함
과적합	매우 높음	낮음 (크게 떨어짐)	기출문제 답을 통째로 외움

진단의 핵심은 학습 성능과 테스트 성능의 차이입니다.

테스트 데이터는 마지막에 딱 한 번

데이터	역할	비유
학습 (train)	모델이 배우는 데이터	교과서, 문제집
검증 (validation)	모델·설정을 고르는 데이터	모의고사
테스트 (test)	최종 성능을 한 번 확인	수능

테스트 데이터를 보며 모델을 고치면 테스트 데이터에 과적합됩니다 (원칙 5, 테스트는 닮는다).

데이터가 적을 때, 시간 순서가 있을 때

데이터가 적으면

검증 데이터를 따로 떼지 않고 교차 검증으로 학습 데이터를 돌아가며 검증에 씁니다. 2일차 4교시에 직접 구현합니다.

시간 순서가 있으면

주가·판매량은 무작위로 섞으면 안 됩니다. 과거로 학습하고 미래로 테스트합니다. 섞으면 미래 정보가 새어 들어갑니다.

결정 트리 깊이와 과적합

최대 깊이	학습 정확도	테스트 정확도	차이	진단
1	0.923	0.923	0.000	단순함
2	0.958	0.909	0.049	
3	0.977	0.944	0.032	
5	0.995	0.951	0.044	테스트 최고
8	1.000	0.937	0.063	과적합
제한 없음	1.000	0.937	0.063	과적합

깊이를 늘리면 학습 정확도는 1.0까지 오르지만 테스트 정확도는 오히려 떨어집니다.

5교시 · 누구로 고를 것인가

테스트로 고르면 5, 교차 검증으로 고르면 3

테스트 데이터로 고름

깊이 5. 이 테스트 143건에 운 좋게 맞은 선택일 수 있습니다. 깊이 1과 2의 순서도 뒤집힐 만큼 들쭉날쭉합니다.

학습 데이터 안에서 5겹 교차 검증

깊이 3. 테스트 데이터를 쓰지 않고 고른, 더 믿을 만한 선택입니다.

원칙 5(테스트는 닮는다)의 실제 사례입니다.

모양을 보면 처방이 보입니다

모양	진단	처방
학습·검증 둘 다 낮고 가까움	과소적합	더 복잡한 모델, 더 좋은 특성
학습 높고 검증 낮으며 간격 큼	과적합	데이터 추가, 모델 단순화, 규제
데이터가 늘수록 간격이 줄어듦	데이터를 더 모으면 좋아질 가능성	데이터 추가

노트북 03에서 validation_curve, learning_curve로 직접 그림니다.

예측 시점에 없는 정보가 학습에 섞이면

유형	예	예방
전처리 누수	전체 데이터로 스케일링한 뒤 분할	분할 먼저, Pipeline으로 학습 데이터만 fit
타깃 누수	연체 예측에 '연체 후 독촉 횟수' 특성	예측 시점에 알 수 있는 값인지 검토
시간 누수	미래 데이터를 섞어 무작위 분할	시간 순서대로 분할
중복 누수	같은 환자가 학습·테스트에 모두	중복 제거, 사람 단위로 분할

테스트 성능은 좋은데 운영에서 무너집니다.

무작위 데이터로 90% 정확도 만들기

특성 10,000개와 라벨이 모두 무작위인 데이터 100건. 제대로 평가하면 0.5 근처여야 합니다.

0.90

누수: 전체 데이터로 특성 20개를 먼저 고른 뒤 교차 검증

0.57

올바름: 특성 선택을 Pipeline 안에서 학습 부분에만

Pipeline이 학습 데이터로만 fit하도록 보장합니다

```
# 나쁜 예: 분할 전에 전체로 fit
X_scaled = StandardScaler().fit_transform(X)
X_train, X_test, ... = train_test_split(X_scaled, y, ...)

# 좋은 예: Pipeline
model = make_pipeline(StandardScaler(), LogisticRegression())
model.fit(X_train, y_train)
```

누수를 의심할 신호: 성능이 너무 좋을 때, 특성 하나의 중요도가 압도적일 때, 운영에서 성능이 급락할 때

예방하는 방법과 진단하는 방법

방법	원리
데이터 추가·증강	외울 수 없을 만큼 다양한 예
모델 단순화	깊이 제한, 특성 수 줄이기
규제 (L1, L2)	가중치가 너무 커지지 않게 벌점
조기 종료 · 드롭아웃	신경망 학습에서 과적합 억제
앙상블	여러 모델의 예측을 평균
교차 검증	과적합을 더 믿을 만하게 진단 (예방이 아님)

선택 5교시 · 편향-분산 트레이드오프

복잡도의 적정점을 찾는 일

편향 (Bias)

너무 단순해서 생기는 체계적 오차 → 과소적합

분산 (Variance)

데이터가 조금만 바뀌어도 모델이 크게 바뀜 → 과적합

주의: 이 '편향'은 통계 용어입니다. 6교시의 데이터 편향(공정성)과 다릅니다.

데이터가 현실을 공정하게 대표하지 못할 때

종류	예
표본(선택) 편향	스마트폰 앱 설문 데이터에 고령층이 적음
역사적 편향	과거 합격자 대부분이 남성 → 채용 AI (1교시 사례)
측정 편향	특정 병원 장비의 영상만 화질이 낮음
라벨 편향	작성자 집단에 따라 같은 문장을 다르게 라벨링
대표성 부족	희귀 질환, 소수 언어 데이터가 너무 적음

전체만 보면 소수 그룹의 문제가 가려집니다

그룹	테스트 건수	정확도	재현율
전체	600	0.880	0.744
A (학습 데이터의 90%)	540	0.893	0.761
B (학습 데이터의 10%)	60	0.767	0.588

가상 데이터. 두 그룹은 양성 비율은 비슷하지만 특성과 결과의 관계가 조금 다릅니다. 모델은 그룹 정보 없이 학습해 A의 규칙만 배웠습니다.

나눠 보고, 같은 지표로 비교하고, 기준을 둔다

1

데이터 확인

민감 속성(성별, 연령대, 지역)별 데이터 수와
라벨 비율

2

그룹별 지표

그룹마다 같은 지표(재현율 등)를 계산

3

차이에 기준

예: 그룹 간 재현율 차이 0.05 이하

이 기준이 7교시 검수 기준서와 2일차 테스트 코드에 들어갑니다.

어떤 공정성을 택할지는 합의의 문제

지표	뜻	예
인구통계적 동등성	그룹마다 양성 판정 비율이 같음	대출 승인율이 성별과 무관
기회 균등	그룹마다 재현율이 같음	상환 능력이 있는 사람의 승인율이 그룹별로 같음

둘을 동시에 만족하기 어려운 경우가 많습니다. 검증 담당자의 역할은 측정하고 드러내는 것입니다.

선택 6교시 · 민감 속성을 지우면 해결될까

대리 변수가 신호를 대신 전합니다

성별 컬럼을 지워도 이름, 특정 활동 이력처럼 성별과 관련된 특성이 신호를 대신 전달할 수 있습니다. 채용 AI 사례가 이 경우입니다. 그래서 속성을 빼는 것만으로는 부족하고, 결과를 그룹별로 측정해야 합니다.

예상 밖 입력에도 안정적으로

입력이 조금 바뀌거나 예상 밖 입력이 들어와도 모델이 안정적으로 올바르게 동작하는 성질. AI 시스템 품질 모델(ISO/IEC 25059)도 강건성을 품질 특성으로 다룹니다.

노이즈

센서 잡음, 흐린 사진, 오타

결측·이상치

빈 값, 나이 999

분포 변화

학습 때와 운영 때 데이터가 다름

적대적 예제

모델을 속이도록 조작된 입력

정답 라벨 없이도 테스트할 수 있습니다

테스트	질문	예	기대
섭동	작은 잡음에 성능이 얼마나 떨어지나?	특성에 표준편차 5% 잡음	하락 0.02 이하
불변성	영향 없어야 할 변화에 예측이 그대로인가?	고객 이름만 변경	예측 불변
방향성	정해진 방향으로 바뀌어야 하는가?	면적 증가 → 가격 예측 상승	예측 증가
범위 밖 입력	학습 범위 밖 값이 들어오면?	나이 150, 음수 소득	거부 또는 경고

불변성·방향성 테스트는 NLP 모델 테스트 방법론 CheckList(Ribeiro 외, 2020)로 널리 알려졌습니다. 2일차에 PyTest로 작성합니다.

잡음과 결측에 대한 강건성

잡음 (표준편차 × 배수)	0	0.05	0.1	0.2	0.5	1.0
정확도 (10회 평균)	0.986	0.981	0.980	0.975	0.955	0.901

결측 비율 (학습 데이터 중앙값으로 대체)	0%	10%	30%	50%
정확도	0.986	0.972	0.965	0.930

방향성 테스트: bmi만 올렸을 때 예측 진행도가 올라간 환자 100% (통과)

선택 6교시 · 분포 변화 감지

운영 중에 데이터가 바뀝니다

데이터 드리프트

입력 분포가 바뀜. 예: 신규 고객층 유입

개념 드리프트

입력과 정답의 관계가 바뀜. 예: 경기 변화로 같은 조건이어도 연체 가능성이 달라짐

감지: 운영 데이터 분포를 학습 데이터와 주기적으로 비교, 성능 지표 지속 모니터링

사람 눈에는 같아 보이는데 모델은 속는다

- 미세한 잡음을 더해 판다 사진을 긴팔원숭이로 분류하게 만든 연구(Goodfellow 외, 2015)가 대표 사례
- 자율주행, 얼굴 인식처럼 보안이 중요한 분야에서 특히 문제
- 이 특강에서는 개념만 소개합니다

7교시 · 데이터 품질 관리

Garbage In, Garbage Out

모델 품질의 상한은 데이터 품질이 정합니다.

여섯 가지 질문

차원	질문	점검 예
완전성	빠진 값은 없나?	컬럼별 결측률
유효성	형식과 범위가 맞나?	나이 0~120, 범주값 목록
정확성	실제와 일치하나?	표본 원본 대조, 라벨 오류 검수
일관성	서로 모순되지 않나?	가입일 < 탈퇴일, 단위 통일
유일성	중복은 없나?	같은 ID·같은 행
적시성	충분히 최신인가?	수집 기간, 마지막 갱신일

데이터 품질 관리 분야에서 널리 쓰는 분류입니다. 문헌마다 이름과 개수가 조금씩 다릅니다.

무엇을, 어떻게 검사하나

항목	검사 방법	pandas
스키마	컬럼 이름·개수·자료형	df.dtypes
결측	컬럼별 결측률	df.isna().mean()
범위	허용 범위 안인가	df["age"].between(0, 120)
범주값	허용된 값만 있나	df["sex"].isin({"M", "F"})
중복	완전 중복 행, ID 중복	df.duplicated().sum()
라벨	분포, 규칙 위반 의심	df["label"].value_counts()
분할 누수	학습·테스트 ID 교집합	set(train) & set(test)

오류 6종류를 섞어 둔 데이터에서 찾아내기

```
checks = [  
    ("결측", "컬럼별 결측률 최댓값", "0.05 이하", df.isna().mean().max()),  
    ("범위", "범위 밖 값 건수", "0건", sum(out_of_range.values())),  
    ("중복", "완전 중복 행", "0건", int(df.duplicated().sum())),  
    ("분할 누수", "학습·테스트 ID 교집합", "0건", len(train_ids & test_ids)),  
    ...  
]
```

찾는 것에서 끝나지 않고, 어떻게 처리할지 정하는 것이 검수 기준입니다.

항목 · 허용 기준 · 벗어나면 조치

영역	기준 예	벗어나면
결측	컬럼별 5% 이하	이하: 중앙값 대체 / 초과: 담당자 검토
범위·범주	범위 밖·허용 외 값 0건	제외 후 목록 전달, 매핑 정정
중복	완전 중복·ID 중복 0건	제거 후 건수 기록
라벨	비율 $\pm 5\%$ p, 표본 검수 오류율 2% 이하	라벨 재검수
분할	층화, ID 교집합 0건, 시계열은 시간순	분할 재실행
전처리	스케일링·대체값은 학습 데이터로만	Pipeline으로 수정

숫자보다 중요한 두 가지

기준은 정황에 따라

5%, $\pm 5\%$ p는 예시입니다. 데이터와 서비스의 위험도에 따라 팀이 정합니다(원칙 6).

기준은 먼저 합의

결과를 보고 나서 기준을 바꾸면 검수의 의미가 없습니다. 사전에 문서로 합의합니다.

TTA 「신뢰할 수 있는 인공지능 개발 안내서」

요구사항	오늘 내용
03 인공지능 시스템의 신뢰성 테스트 계획 수립	테스트 프로세스, 검수 기준서
05 데이터의 활용을 위한 상세 정보 제공	출처, 수집 기간, 전처리 내역 기록
06 데이터 견고성 확보를 위한 이상 데이터 점검	결측·범위·이상치 검사
07 수집 및 가공된 학습 데이터의 편향 제거	그룹별 분포와 성능 점검 (6교시)

과학기술정보통신부·TTA가 발간한 안내서의 15개 요구사항 중 데이터·테스트 관련 항목

도구보다 기준이 먼저입니다

도구	특징
pandas + assert	가장 단순. 오늘과 2일차 실습 방식
pandera	데이터프레임 스키마와 규칙을 코드로 선언해 검증
Great Expectations	데이터에 대한 기대를 정의하고 검증 결과를 문서로 생성
DVC	큰 데이터 파일의 버전을 Git과 함께 추적

최소한 '어떤 데이터로 학습했는지'는 반드시 기록합니다. 재현할 수 있어야 버그를 고칩니다.

8교시 · 조별 실습 50분

두 가지 과제

시간	활동
20분	과제 A: 노트북 05 결과로 데이터 품질 검수 기준서 작성
10분	과제 B: 사기 탐지 모델 두 개의 평가 결과 해석
15분	조별 발표 (조당 2~3분)
5분	정리와 2일차 예고

템플릿: ai-quality-lab / day1 / templates / data-quality-checklist.md

데이터 품질 검수 기준서

- 1

기준 조정

노트북의 허용 기준이 적절한지 조별로
토론하고 조정
- 2

조치 결정

기준을 벗어난 항목마다 삭제·대체·원본
확인 중 선택
- 3

종합 판정

학습 진행 가능 / 조건부 가능 / 불가와 근거

사기 탐지 모델, 어느 쪽을 고를까 (테스트 10,000건 중 사기 100건)

모델	TP	FP	FN	TN
모델 1	60	40	40	9,860
모델 2	85	400	15	9,500

- 두 모델의 정확도, 정밀도, 재현율, F1을 계산하세요
- 은행 입장에서 어떤 모델을 고르겠습니까? 근거는?
- 출시 전에 추가로 확인해야 할 것 두 가지는? (힌트: 5·6교시)

2일차 예고

오늘의 기준을 코드로 자동화합니다

- unittest · PyTest로 전처리 함수와 데이터 검증 테스트
- 교차 검증과 성능 리포트 자동 생성
- GitHub Actions로 자동 테스트, 이슈로 버그 리포트
- 준비물: GitHub 계정 로그인 확인, Google 계정